UNIBALSAS Centro Universitário

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

UNIBALSAS – CENTRO UNIVERSITÁRIO DE BALSAS

CARINNE DA SILVA BORGES

JOÃO PEDRO STRASSER SANTOS

JÚLIO CÉSAR LIMA DOS REIS

ROGÉRIO MOTA DE MELO

BALSAS EXPERIENCE API:
API REST PARA ATRAÇÕES TURÍSTICAS DE BALSAS-MA
RELATÓRIO DE ENTREGA DO PROJETO

Balsas – MA

2026

UNIBALSAS Centro Universitário

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

CARINNE DA SILVA BORGES
JOÃO PEDRO STRASSER SANTOS
JÚLIO CÉSAR LIMA DOS REIS
ROGÉRIO MOTA DE MELO

BALSAS EXPERIENCE API:
API REST PARA ATRAÇÕES TURÍSTICAS DE BALSAS-MA
RELATÓRIO DE ENTREGA DO PROJETO

Trabalho de laboratório do curso de Análise e Desenvolvimento de Sistemas do Centro Universitário de Balsas – UNIBALSAS. Apresenta a construção de uma API REST em Node.js e Express, integrada ao PostgreSQL com Prisma ORM.

Disciplina: Programação para Web.
Professor: Marcos David de Souza Ramos.

Balsas – MA
2026

1 IDENTIFICAÇÃO

Campo	Informação
Instituição	UNIBALSAS - Centro Universitário de Balsas
Curso	Análise e Desenvolvimento de Sistemas
Disciplina	Programação para Web
Professor	Marcos David de Souza Ramos
Projeto	Balsas Experience API
Data do documento	17/10/2026

2 A EQUIPE

Equipe: **Carinne, Júlio César, João Pedro e Rogério**.

Distribuição de atividades confirmada pela equipe.

Nome completo	Matrícula	Contribuição
Carinne da Silva Borges	11.1.01227	Organização do README, descrição dos endpoints e seleção das evidências de funcionamento.
Júlio César Lima dos Reis	25.1.07087	Desenvolvimento das rotas de atrações e avaliações, com validação dos dados recebidos.
João Pedro Strasser Santos	25.1.04381	Modelagem das entidades, migrações e integração das consultas com Prisma e PostgreSQL.
Rogério Mota de Melo	25.1.04936	Configuração do ambiente, testes de integração e publicação da API no Render.

3 DESCRIÇÃO DO PROJETO

A Balsas Experience API disponibiliza um catálogo de atrações turísticas de Balsas-MA. Permite consultar atrações, filtrar por categoria, tag e texto, acessar detalhes e registrar avaliações de visitantes. Também oferece operações para criar, atualizar e remover atrações.

Serve como fonte de dados para sites e aplicativos voltados a moradores e turistas. O projeto reúne informações dispersas em uma estrutura consultável e persistente: categorias organizam o catálogo, tags permitem cruzar interesses e avaliações registram a experiência dos visitantes.

4 TECNOLOGIAS UTILIZADAS

Versões de referência do ambiente local e dependências. A versão do banco publicada não foi consultada.

Tecnologia	Versão verificada	Finalidade
Node.js	24.13.1	Ambiente de execução JavaScript
Express	5.2.1	Rotas e respostas HTTP
Prisma CLI / Client	7.10.0	Modelagem, migrações e consultas
PostgreSQL	18.6	Banco de dados relacional
@prisma/adapter-pg	7.10.0	Adaptador Prisma para PostgreSQL
pg	8.23.1	Driver PostgreSQL
dotenv	18.0.5	Carregamento das variáveis de ambiente
npm	11.17.0	Dependências e scripts

5 MODELO DE DADOS

PK = chave primária; FK = chave estrangeira. IDs inteiros são gerados automaticamente; campos sem indicação de opcional são obrigatórios.

Tabela	Campos e restrições
Categoria	id: Int, PK; nome: String, único
Tag	id: Int, PK; nome: String, único
Atracao	id: Int, PK; nome: String; descricao: String; cidade: String (padrão Balsas); estado: String (padrão MA); distanciaCentroKm: Int opcional; criadaEm: DateTime (agora); categoriaId: Int, FK → Categoria.id

Tabela	Campos e restrições
Avaliacao	id: Int, PK; visitante: String; nota: Int; comentario: String opcional; criadaEm: DateTime (agora); atracaoId: Int, FK → Atracao.id
_AtracaoToTag	A: Int, FK → Atracao.id; B: Int, FK → Tag.id. PK composta (A, B). Tabela de associação gerenciada pelo Prisma.

Relações: Categoria 1:N Atracao; Atracao 1:N Avaliacao; Atracao N:N Tag. Excluir uma atração remove suas avaliações e associações com tags em cascata; categorias e tags são preservadas. A API valida notas inteiras entre 1 e 5.

Por que separar tags? Uma tabela própria permite reutilizar a mesma tag em várias atrações, evitar duplicação de nomes e aplicar filtros por relacionamento.

6 ENDPOINTS DA API

Método	Caminho	O que faz	Status
GET	/	Identificação do serviço	200
GET	/health	Estado do processo HTTP	200
GET	/atracoes	Listagem por nome; categoria e tags incluídas	200
GET	/atracoes?categoria=Lazer&tag=lazer&q=rio	Filtros combinados, sem diferenciar maiúsculas	200
GET	/atracoes/:id	Detalhes, avaliações, média e total	200 / 404
GET	/categorias	Categorias e contagem de atrações	200
POST	/atracoes	Criar atração	201 / 400 / 409
PUT	/atracoes/:id	Substituir dados e tags	200 / 400 / 404 / 409
DELETE	/atracoes/:id	Excluir atração e avaliações	204 / 404
POST	/atracoes/:id/avaliacoes	Criar avaliação de visitante	201 / 400 / 404

Falhas inesperadas podem retornar 500. /health não verifica o banco. POST e PUT exigem nome, descricao e categoria. Cidade, estado, distância e tags são opcionais. PUT substitui tags e restaura os padrões dos campos opcionais omitidos.

6 ENDPOINTS DA API · EXEMPLO COMPLETO

Requisição e resposta reais do cliente HTTP Node.js/fetch, capturadas em ambiente local em 15/10/2026. O registro temporário foi removido após a coleta.

Requisição

```
POST /atracoes HTTP/1.1
Host: 127.0.0.1:3110
Content-Type: application/json

{
  "nome": "Evidência de entrega 1792081549947",
  "descricao": "Registro temporário para verificar a entrega acadêmica.",
  "categoria": "Histórico e Urbano",
  "cidade": "Balsas",
  "estado": "MA",
  "distanciaCentroKm": 0,
  "tags": []
}
```

Resposta

```
HTTP/1.1 201 Created
Location: /atracoes/11
Content-Type: application/json; charset=utf-8

{
  "id": 11,
  "nome": "Evidência de entrega 1792081549947",
  "descricao": "Registro temporário para verificar a entrega acadêmica.",
  "cidade": "Balsas",
  "estado": "MA",
  "distanciaCentroKm": 0,
  "criadaEm": "2026-10-15T16:25:50.047Z",
  "categoriaId": 1,
  "categoria": {
    "id": 1,
    "nome": "Histórico e Urbano"
  },
  "tags": []
}
```

Busca inexistente: `GET /atracoes/2147483647` retorna 404 com `{"erro":"Atração não encontrada"}`. Remoção bem-sucedida retorna 204 sem corpo.

7 COMO EXECUTAR O PROJETO

Pré-requisitos: Node.js 24 LTS, npm, Git e acesso a um PostgreSQL local ou hospedado. Execute os comandos na raiz do repositório.

1. Clonar e instalar

```
git clone https://github.com/ark-pkr/balsas-experience-api.git
cd balsas-experience-api
npm ci
```

O repositório público está indicado na seção 9.

2. Configurar o ambiente

Copie .env.example para .env e substitua os exemplos pelas credenciais do seu banco. Não publique o .env.

```
DATABASE_URL="postgresql://USUARIO:SENHA_FICTICIA@HOST-POOLER/neondb?sslmode=require"
DIRECT_URL="postgresql://USUARIO:SENHA_FICTICIA@HOST-DIRETO/neondb?
sslmode=require&connect_timeout=30"
PORT=3000
```

A aplicação usa DATABASE_URL. O Prisma CLI usa DIRECT_URL ou, se ausente, DATABASE_URL. As senhas acima são fictícias.

3. Gerar cliente, migrar e popular

```
npx prisma generate --config prisma7.config.ts
npx prisma migrate deploy --config prisma7.config.ts
npm run seed
```

Atenção ao seed: apaga os dados existentes antes de inserir oito atrações e três avaliações. Use apenas em banco novo ou descartável; pule essa etapa se o banco já estiver populado.

4. Iniciar e verificar

```
npm start
# Em desenvolvimento, alternativamente: npm run dev
# Em outro terminal:
curl -i http://localhost:3000/health
curl -i http://localhost:3000/atracoes
npm test
```

No PowerShell, use curl.exe. Os testes usam a porta 3108 e criam/removem uma atração temporária. Os 15 cenários passaram na verificação local.

Publicação preparada

O arquivo render.yaml configura um Web Service no Render com Node.js 24.13.1. Build: `npm ci --include=dev && npm run build`; início: `npm start`. Cadastrar DATABASE_URL e DIRECT_URL no painel. O build aplica migrações e não executa seed. A hospedagem fornece PORT.

8. EVIDÊNCIAS DE FUNCIONAMENTO

Listagem de atrações · HTTP 200

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.

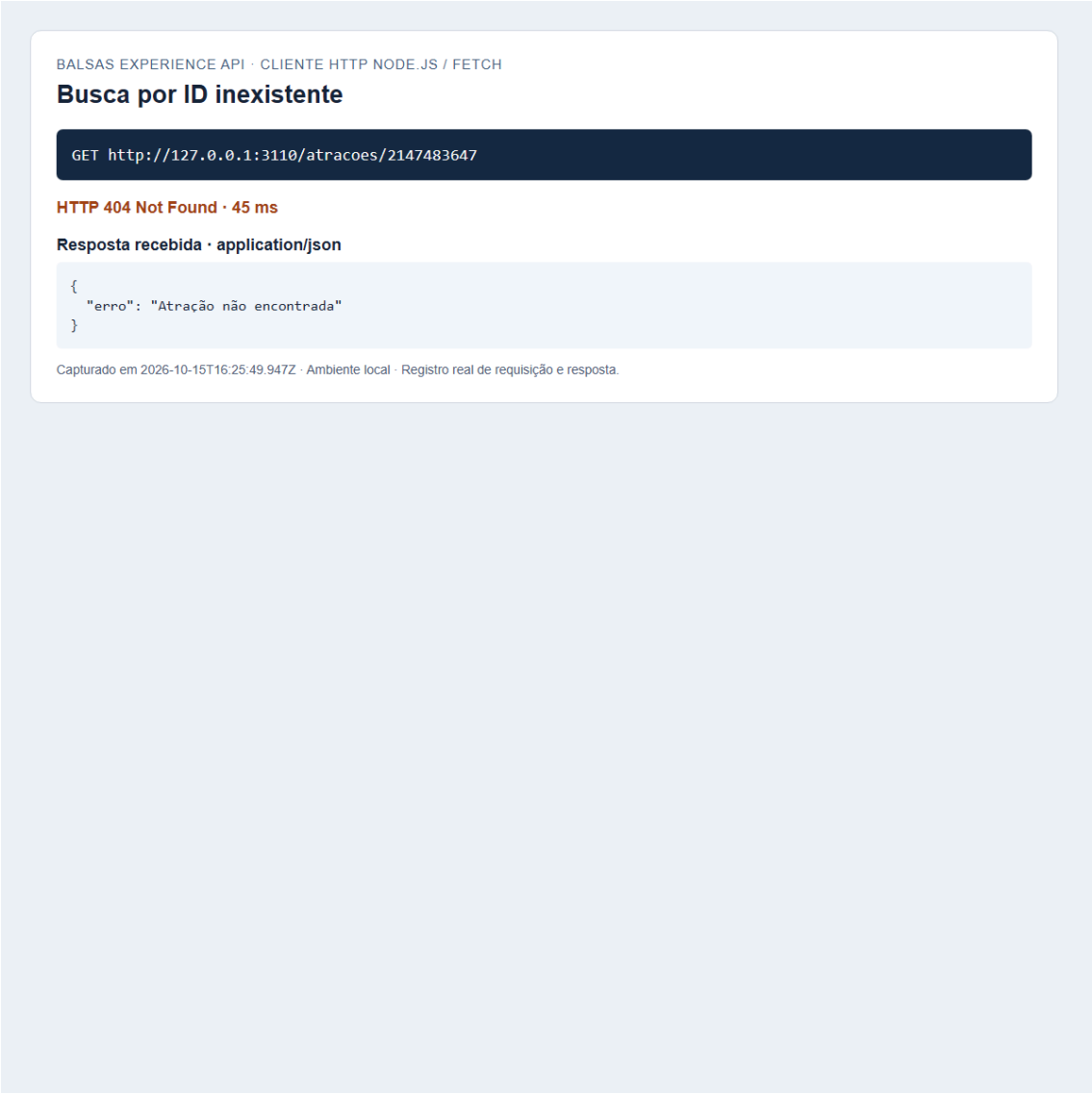


GET http://127.0.0.1:3110/atracoes · Status recebido: 200. A captura mostra o primeiro item da listagem; o JSON integral foi preservado.

8. EVIDÊNCIAS DE FUNCIONAMENTO · continuação

Busca por ID inexistente · HTTP 404

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.

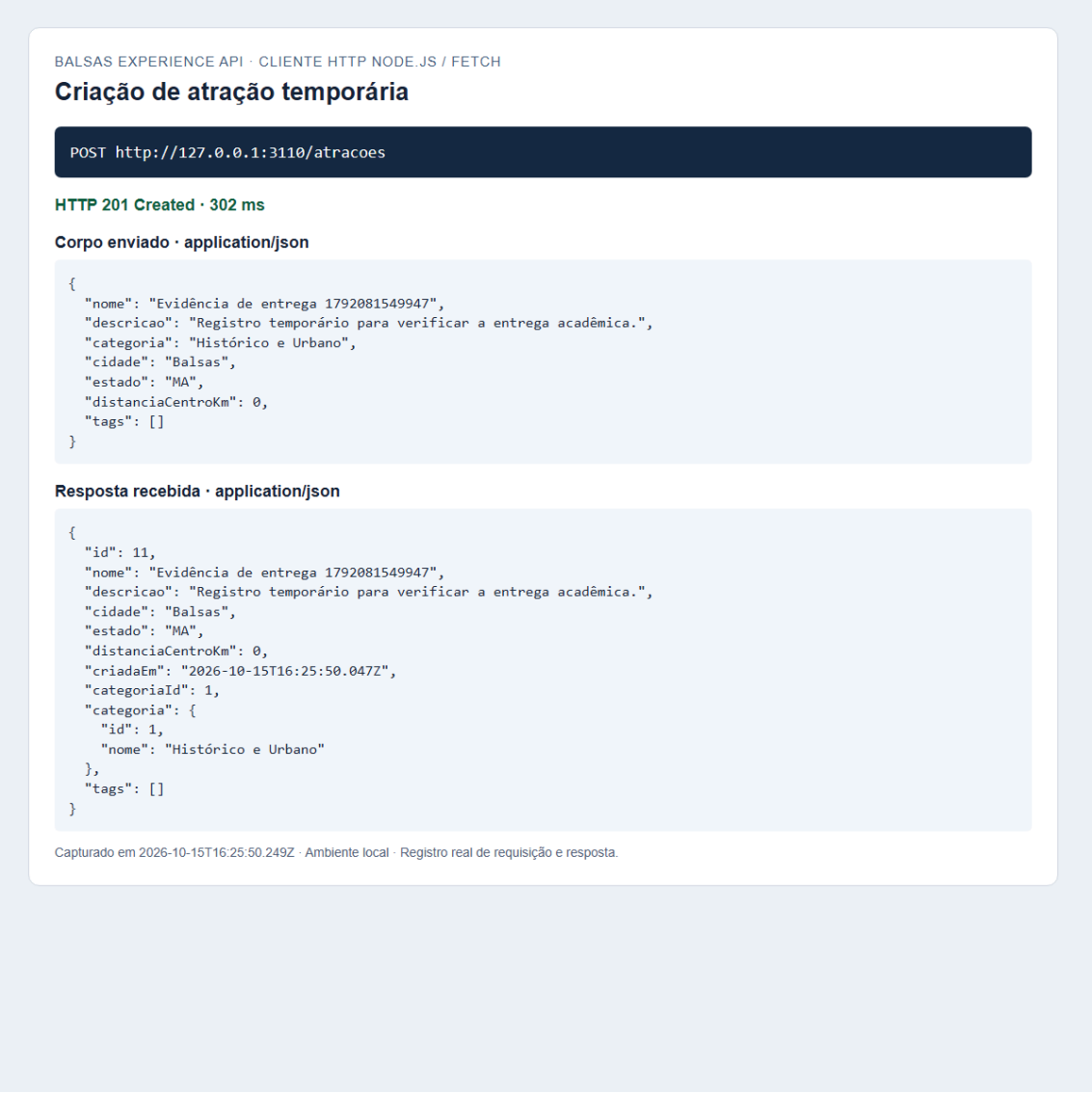


GET http://127.0.0.1:3110/atracoes/2147483647 · Status recebido: 404.

8. EVIDÊNCIAS DE FUNCIONAMENTO · continuação

Criação de atração temporária · HTTP 201

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.

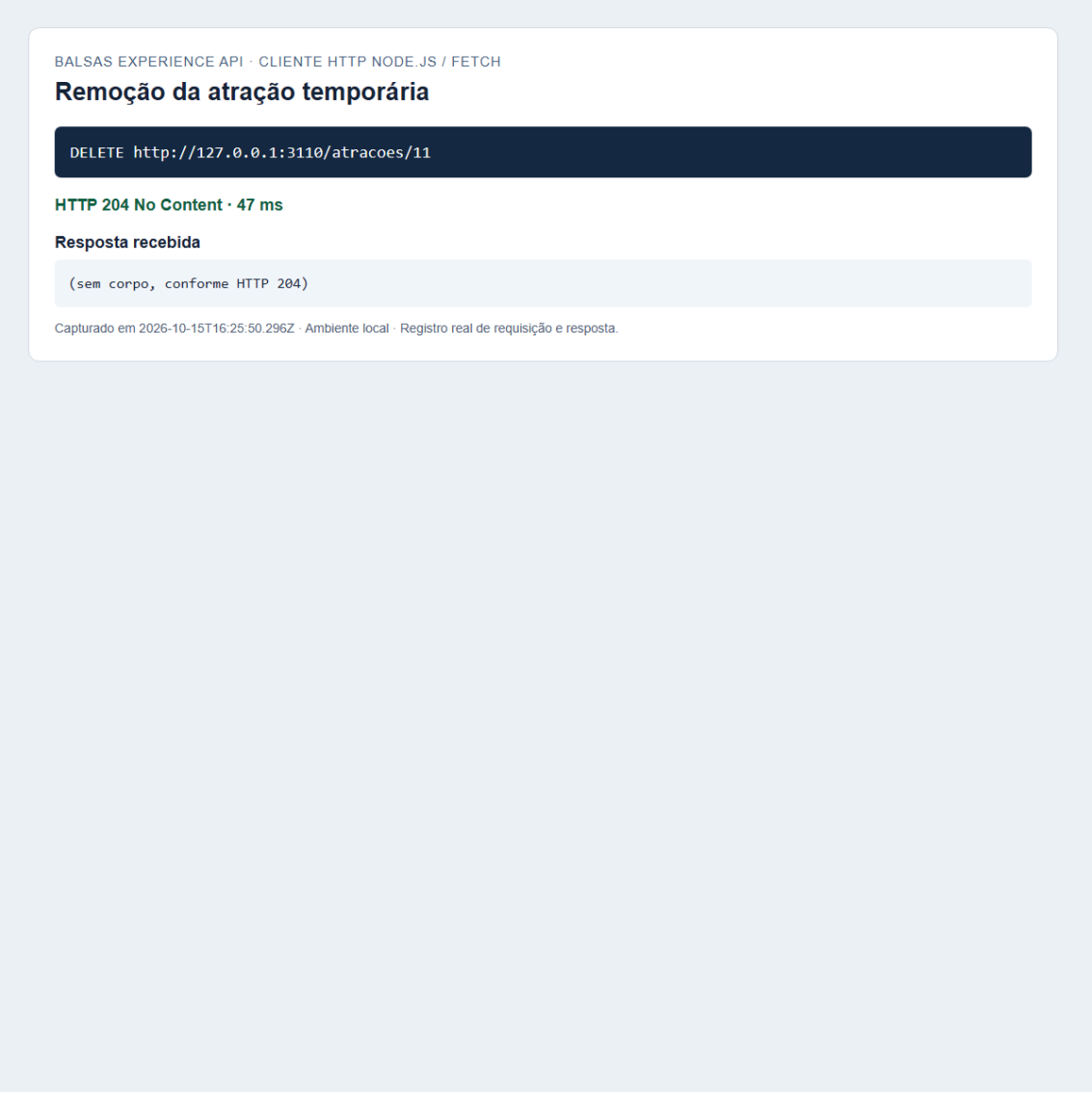


POST http://127.0.0.1:3110/atracoes · Status recebido: 201.

8. EVIDÊNCIAS DE FUNCIONAMENTO · continuação

Remoção da atração temporária · HTTP 204

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.



DELETE http://127.0.0.1:3110/atracoes/11 · Status recebido: 204.

8. EVIDÊNCIAS DE FUNCIONAMENTO · API publicada

Verificação real em 17/10/2026: GET /health, /categorias e /atracoes retornaram HTTP 200; GET /atracoes/2147483647 retornou 404. As consultas de categorias e atrações confirmam o acesso ao banco publicado.



Captura da visualização da resposta real do cliente HTTP Node.js/fetch. Registros integrais: docs/evidencias/registros-publicos.json.

9. LINKS

Destino	Endereço
Repositório público no GitHub	https://github.com/ark-pkr/balsas-experience-api
API publicada	https://balsas-experience-api-lgdm.onrender.com/categorias

10 DIFICULDADES E APRENDIZADOS

Relatos individuais e contribuições revisados e confirmados pela equipe.

Integrante	Dificuldade	Solução	Aprendizado
Carinne da Silva Borges	Distinguir os códigos 200, 201, 204 e 404 na documentação.	Comparação das requisições e respostas para registrar método, caminho, corpo e status de cada operação.	Documentar uma API de forma reproduzível, incluindo exemplos e instruções de execução.
Júlio César Lima dos Reis	Tratar campos obrigatórios, notas inválidas e registros inexistentes.	Organização dos controladores e verificação dos retornos HTTP em entradas válidas e inválidas.	Implementar operações CRUD no Express, validar JSON e utilizar códigos de status adequados.
João Pedro Strasser Santos	Representar categorias, tags reutilizáveis e avaliações sem duplicar informações.	Definição de relações 1:N e N:N no schema, com chaves estrangeiras e exclusão em cascata das avaliações.	Usar migrações, consultar relações com Prisma e preservar a integridade dos dados.
Rogério Mota de Melo	Conectar a aplicação ao banco hospedado e configurar as variáveis de ambiente.	Configuração de DATABASE_URL, DIRECT_URL e comandos de build, seguida de testes das rotas publicadas.	Publicar uma API, proteger credenciais e verificar o funcionamento do servidor e do banco.